

UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

# ALGORITMOS DE CAMINOS MAS CORTOS

CIENCIAS DE LA COMPUTACIÓN II

Luisa Fernanda Guerrero Ordoñez-20212020099
Juan David Quiroga Gomez-20222020206
David Santiago Lugo Piñeros-20222020180

Bogotá D.C.

2026

# Índice

**1. Introducción** **3**

**2. Objetivos** **3**
2.1. Objetivo general . . . . . 3
2.2. Objetivos específicos . . . . . 3

**3. Marco teórico** **3**
3.1. Algoritmo de Dijkstra . . . . . 3
3.2. Algoritmo de Bellman-Ford . . . . . 4
3.3. Algoritmo de Floyd-Warshall . . . . . 4

**4. Casos de uso de cada algoritmo** **4**
4.1. Dijkstra . . . . . 4
4.2. Bellman-Ford . . . . . 5
4.3. Floyd-Warshall . . . . . 5

**5. Órdenes de complejidad** **5**

**6. Metodología del laboratorio** **5**
6.1. Pruebas . . . . . 5
6.2. Variables a medir . . . . . 6

**7. Fragmentos de código** **6**
7.1. Código de Dijkstra . . . . . 6
7.2. Código de Bellman-Ford . . . . . 7
7.3. Código de Floyd-Warshall . . . . . 7

**8. Análisis de eficiencia** **8**
8.1. Eficiencia de Dijkstra . . . . . 8
8.2. Eficiencia de Bellman-Ford . . . . . 8
8.3. Eficiencia de Floyd-Warshall . . . . . 8

**9. Comparación general** **9**

**10. Resultados esperados** **9**

**11. Discusión** **9**

**12. Conclusiones** **9**

# 1. Introducción

Los algoritmos de caminos más cortos son fundamentales en el área de estructuras de datos, en teoría de grafos y diseño de algoritmos ya que nos permiten determinar la ruta de menor costo entre nodos dentro de una red. Estos algoritmos tienen aplicaciones en navegación GPS, redes de computadores, sistemas logísticos, telecomunicaciones, videojuegos y muchas otras áreas.

En este laboratorio realizamos un análisis de eficiencia de tres algoritmos clásicos de caminos más cortos vistos en clase: **Dijkstra**, **Bellman-Ford** y **Floyd-Warshall**. El propósito es comparar sus casos de uso, sus órdenes de complejidad y su eficiencia dependiendo de las características del problema, como el tipo de pesos en el grafo, la cantidad de nodos y el tipo de consulta requerida.

# 2. Objetivos

## 2.1. Objetivo general

Analizar y comparar la eficiencia de los algoritmos Dijkstra, Bellman-Ford y Floyd-Warshall, identificando sus casos de uso, órdenes de complejidad y comportamiento frente a diferentes tipos de grafos.

## 2.2. Objetivos específicos

- Comprender el funcionamiento general de cada algoritmo.
- Identificar en qué situaciones conviene utilizar cada uno.
- Comparar sus complejidades.
- Evaluar la eficiencia de cada algoritmo según el tipo de grafo.
- Presentar fragmentos de código que permitan mostrar su implementación.

# 3. Marco teórico

Un grafo es una estructura compuesta por un conjunto de nodos o vértices y un conjunto de aristas que conectan pares de nodos. En muchos problemas, cada arista tiene asociado un peso, que puede representar distancia, costo, tiempo, energía, o cualquier otra medida.

El problema de caminos más cortos consiste en encontrar la ruta con menor costo entre un nodo origen y un nodo destino, o incluso entre todos los pares de nodos.

## 3.1. Algoritmo de Dijkstra

El algoritmo de Dijkstra permite encontrar el camino más corto desde un nodo origen hacia todos los demás nodos del grafo, siempre que los pesos de las aristas sean no negativos.

Su idea principal consiste en seleccionar de manera iterativa el nodo con menor distancia conocida, y luego actualizar las distancias de sus vecinos.

**Ventajas:**

- Es muy eficiente en grafos con pesos positivos.
- Es ampliamente usado en aplicaciones reales como GPS y redes.

**Desventajas:**

- No funciona correctamente si hay pesos negativos.

## 3.2. Algoritmo de Bellman-Ford

El algoritmo de Bellman-Ford también encuentra caminos más cortos desde un nodo origen hacia todos los demás nodos, pero tiene la ventaja de soportar pesos negativos.

Su funcionamiento se basa en relajar repetidamente todas las aristas del grafo durante $|V| - 1$ iteraciones, donde $V$ es el número de vértices.

**Ventajas:**

- Soporta aristas con pesos negativos.
- Permite detectar ciclos negativos.

**Desventajas:**

- Es más lento que Dijkstra.

## 3.3. Algoritmo de Floyd-Warshall

El algoritmo de Floyd-Warshall calcula los caminos más cortos entre todos los pares de nodos del grafo.

Se basa en programación dinámica y evalúa si pasar por un nodo intermedio mejora la distancia entre dos nodos.

**Ventajas:**

- Encuentra todas las distancias mínimas entre todos los pares.
- Es útil cuando se necesitan muchas consultas de rutas.

**Desventajas:**

- Tiene un costo computacional alto para grafos grandes.

# 4. Casos de uso de cada algoritmo

## 4.1. Dijkstra

Se recomienda utilizar Dijkstra en los siguientes casos:

- Cuando el grafo tiene únicamente pesos positivos o cero.
- Cuando se necesita hallar la ruta más corta desde un solo nodo origen.
- En sistemas de navegación, rutas urbanas y redes de transporte.

**Ejemplo:** calcular la ruta más rápida desde una estación de origen hacia todas las demás estaciones de una ciudad.

### 4.2. Bellman-Ford

Se recomienda utilizar Bellman-Ford cuando:

- Existen pesos negativos en el grafo.
- Se requiere detectar ciclos negativos.
- Se modelan escenarios financieros, descuentos, penalizaciones o ganancias.

**Ejemplo:** análisis de una red de costos donde algunas conexiones representan reducciones de costo.

### 4.3. Floyd-Warshall

Se recomienda utilizar Floyd-Warshall cuando:

- Se necesita conocer la distancia mínima entre todos los pares de nodos.
- El grafo es pequeño o mediano.
- Se van a realizar muchas consultas de rutas entre distintos pares.

**Ejemplo:** calcular las distancias mínimas entre todas las ciudades de una red vial.

## 5. Órdenes de complejidad

Cuadro 1: Comparación de complejidad de los algoritmos

| **Algoritmo** | **Tipo de problema** | **Complejidad temporal** | **Complejidad espacial** |
|---|---|---|---|
| Dijkstra | Un origen a todos los nodos | $O((V+E)\log V)$ | $O(V+E)$ |
| Bellman-Ford | Un origen a todos los nodos | $O(VE)$ | $O(V)$ |
| Floyd-Warshall | Todos contra todos | $O(V^3)$ | $O(V^2)$ |

## 6. Metodología del laboratorio

Para evaluar la eficiencia de los algoritmos realizamos pruebas sobre distintos tipos de grafos.

### 6.1. Pruebas

- Grafos pequeños: entre 5 y 10 nodos.
- Grafos medianos: entre 50 y 100 nodos.
- Grafos grandes: 500 nodos o más.

- Grafos con pesos positivos.
- Grafos con pesos negativos.
- Grafos densos y grafos dispersos.

### 6.2. Variables a medir

- Tiempo de ejecución.
- Número de nodos.
- Número de aristas.
- Tipo de pesos.
- Uso de memoria.
- Correctitud del resultado.

## 7. Fragmentos de código

A continuación se presentan fragmentos de código en Python para ilustrar la implementación básica de cada algoritmo.

### 7.1. Código de Dijkstra

Listing 1: Implementación básica de Dijkstra en Python

```
import heapq

def dijkstra(grafo, inicio):
    distancias = {nodo: float('inf') for nodo in grafo}
    distancias[inicio] = 0
    cola = [(0, inicio)]

    while cola:
        distancia_actual, nodo_actual = heapq.heappop(cola)

        if distancia_actual > distancias[nodo_actual]:
            continue

        for vecino, peso in grafo[nodo_actual]:
            distancia = distancia_actual + peso
            if distancia < distancias[vecino]:
                distancias[vecino] = distancia
                heapq.heappush(cola, (distancia, vecino))

    return distancias
```

```
grafo = {
    'A': [('B', 4), ('C', 2)],
    'B': [('C', 5), ('D', 10)],
    'C': [('E', 3)],
    'D': [('F', 11)],
    'E': [('D', 4)],
    'F': []
}

print(dijkstra(grafo, 'A'))
```

## 7.2. Código de Bellman-Ford

Listing 2: Implementación básica de Bellman-Ford en Python

```
def bellman_ford(vertices, aristas, inicio):
    distancias = {v: float('inf') for v in vertices}
    distancias[inicio] = 0

    for _ in range(len(vertices) - 1):
        for u, v, peso in aristas:
            if distancias[u] != float('inf') and distancias[u] + peso < dis
                distancias[v] = distancias[u] + peso

    for u, v, peso in aristas:
        if distancias[u] != float('inf') and distancias[u] + peso < distanc
            raise ValueError("El grafo contiene un ciclo negativo")

    return distancias

vertices = ['A', 'B', 'C', 'D']
aristas = [
    ('A', 'B', 1),
    ('A', 'C', 4),
    ('B', 'C', -2),
    ('C', 'D', 2),
    ('B', 'D', 5)
]

print(bellman_ford(vertices, aristas, 'A'))
```

## 7.3. Código de Floyd-Warshall

Listing 3: Implementación básica de Floyd-Warshall en Python

```
def floyd_warshall(matriz):
    n = len(matriz)
    dist = [fila[:] for fila in matriz]
```

```
    for k in range(n):
        for i in range(n):
            for j in range(n):
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])

    return dist

INF = float('inf')

matriz = [
    [0,   3,   INF, 7],
    [8,   0,   2,   INF],
    [5,   INF, 0,   1],
    [2,   INF, INF, 0]
]

resultado = floyd_warshall(matriz)

for fila in resultado:
    print(fila)
```

# 8. Análisis de eficiencia

## 8.1. Eficiencia de Dijkstra

Dijkstra es el algoritmo más eficiente de los tres cuando el grafo no contiene pesos negativos. Su rendimiento mejora bastante cuando se implementa con una cola de prioridad. Por esta razón, es una de las soluciones más utilizadas en problemas reales de rutas.

En grafos grandes y dispersos, su comportamiento es especialmente bueno, ya que evita realizar más operaciones de las necesarias.

## 8.2. Eficiencia de Bellman-Ford

Bellman-Ford es menos eficiente en tiempo de ejecución porque necesita recorrer todas las aristas varias veces. Aun así, su principal fortaleza es la capacidad de manejar pesos negativos y detectar ciclos negativos, algo que Dijkstra no puede hacer.

Por eso, aunque es más lento, sigue siendo importante en problemas donde la estructura del grafo es más compleja.

## 8.3. Eficiencia de Floyd-Warshall

Floyd-Warshall es útil cuando se necesitan todas las distancias mínimas entre todos los pares de nodos. Sin embargo, su complejidad $O(V^3)$ hace que su uso sea costoso cuando la cantidad de nodos aumenta demasiado.

Es una buena opción en grafos pequeños y medianos, o cuando se necesitan muchas consultas de distancias después de un solo procesamiento inicial.

# 9. Comparación general

Cuadro 2: Comparación general entre algoritmos

| **Criterio** | **Dijkstra** | **Bellman-Ford** | **Floyd-Warshall** |
|---|---|---|---|
| Soporta pesos negativos | No | Sí | Sí |
| Detecta ciclos negativos | No | Sí | Sí |
| Fuente única | Sí | Sí | No exclusivamente |
| Todos contra todos | No | No | Sí |
| Rapidez en pesos positivos | Alta | Media | Baja |
| Uso de memoria | Medio | Bajo/medio | Alto |
| Escalabilidad | Alta | Media | Baja en grafos grandes |

# 10. Resultados esperados

A partir de la ejecución de pruebas, se espera encontrar lo siguiente:

- Dijkstra tendrá el menor tiempo de ejecución en grafos con pesos positivos.
- Bellman-Ford será más lento, pero funcionará correctamente con pesos negativos.
- Floyd-Warshall será útil cuando se necesiten distancias entre todos los pares de nodos.
- En grafos grandes, Floyd-Warshall tendrá un costo computacional mucho mayor que los otros dos algoritmos.

# 11. Discusión

Los tres algoritmos resuelven problemas de caminos más cortos, pero no todos se adaptan igual a cualquier situación. Desde el punto de vista práctico, la elección depende del tipo de datos y del objetivo del problema.

Si el interés está en hallar la ruta más corta desde un solo origen y no hay pesos negativos, Dijkstra suele ser la mejor alternativa. Si existen pesos negativos, Bellman-Ford es más adecuado. Si el objetivo es conocer las rutas mínimas entre todos los pares de nodos, Floyd-Warshall es la mejor opción, aunque su costo sea mayor.

En otras palabras, la eficiencia no depende solo de cuál algoritmo tenga menor complejidad, sino también de si sus condiciones de uso coinciden con el problema real.

# 12. Conclusiones

1. Dijkstra es el algoritmo más eficiente para grafos con pesos no negativos.

2. Bellman-Ford es más lento, pero permite trabajar con pesos negativos y detectar ciclos negativos.

3. Floyd-Warshall resulta muy útil cuando se requiere calcular caminos mínimos entre todos los pares de nodos.

4. No existe un algoritmo universalmente mejor, ya que cada uno responde mejor a un tipo de necesidad.

5. La elección correcta del algoritmo depende del tamaño del grafo, del tipo de pesos y de la cantidad de consultas requeridas.